

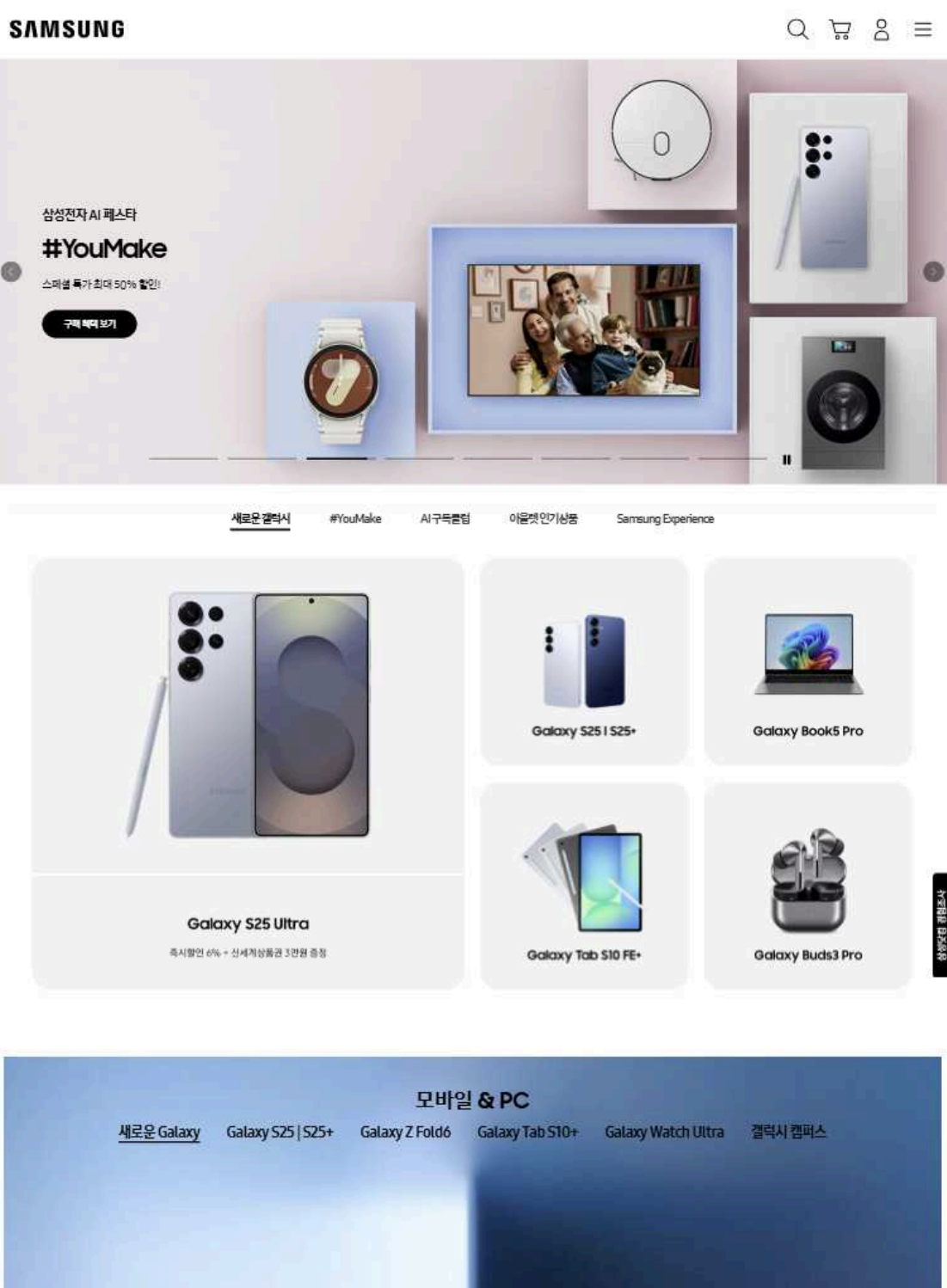
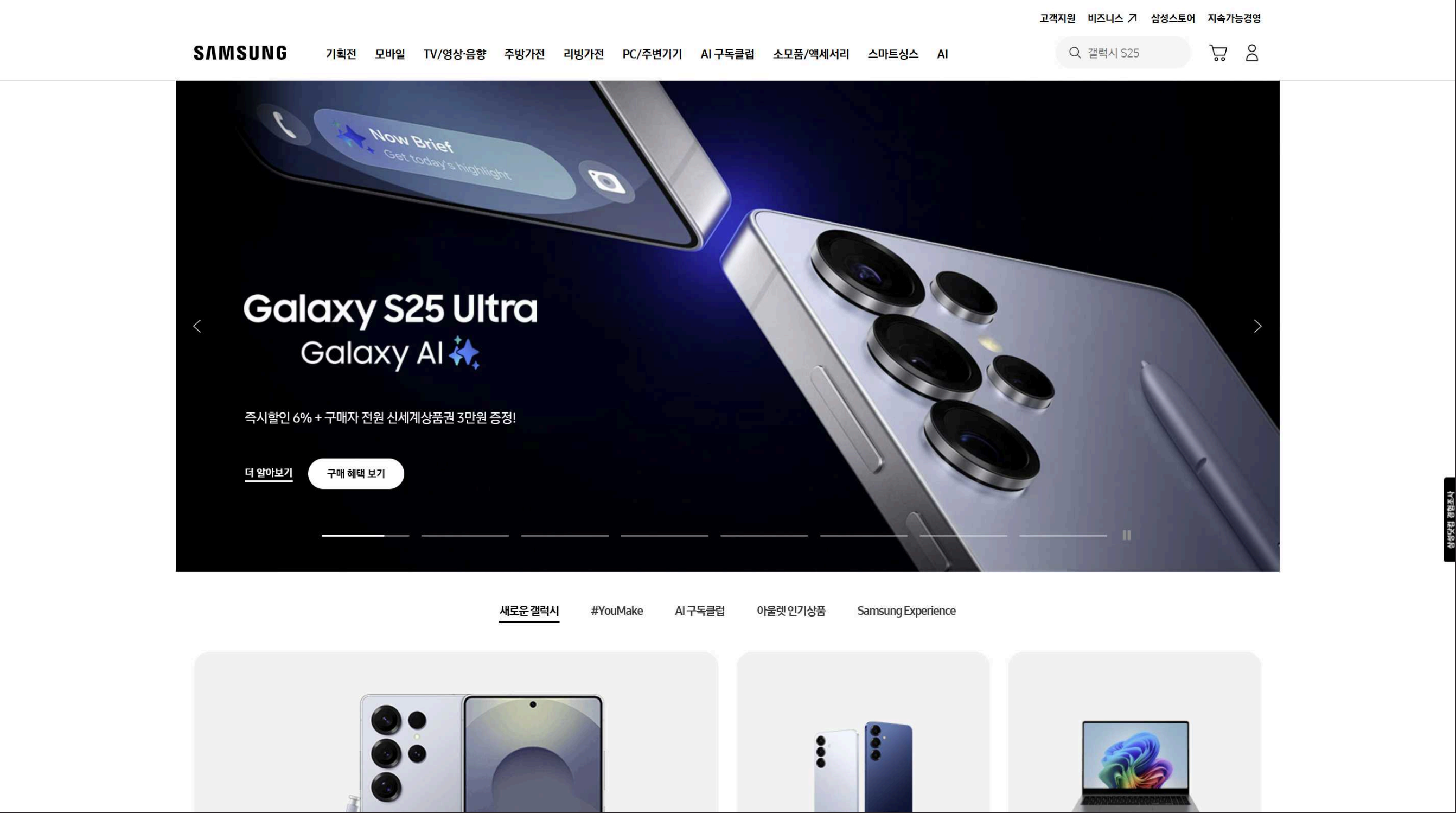
반응형 웹과 공통 컴포넌트

반응형 웹이란?

반응형 웹 (Responsive Web)

- 하나의 웹 사이트에서 PC, 모바일, 태블릿 등 접속하는 디스플레이 종류에 따라 화면의 크기가 자동으로 변하도록 만든 웹
- 디바이스에 맞는 해상도에 따라서 레이아웃과 스타일에 최적화

PC, 태블릿, 모바일



반응형, 왜 필요할까?

사용자 경험 (UX) 향상

모바일과 태블릿을 통해
웹사이트에 접속 → 화면 크기에
관계없이 편리하게 콘텐츠를
탐색할 수 있도록 하여
긍정적인 경험을 제공

SEO (검색 엔진 최적화)

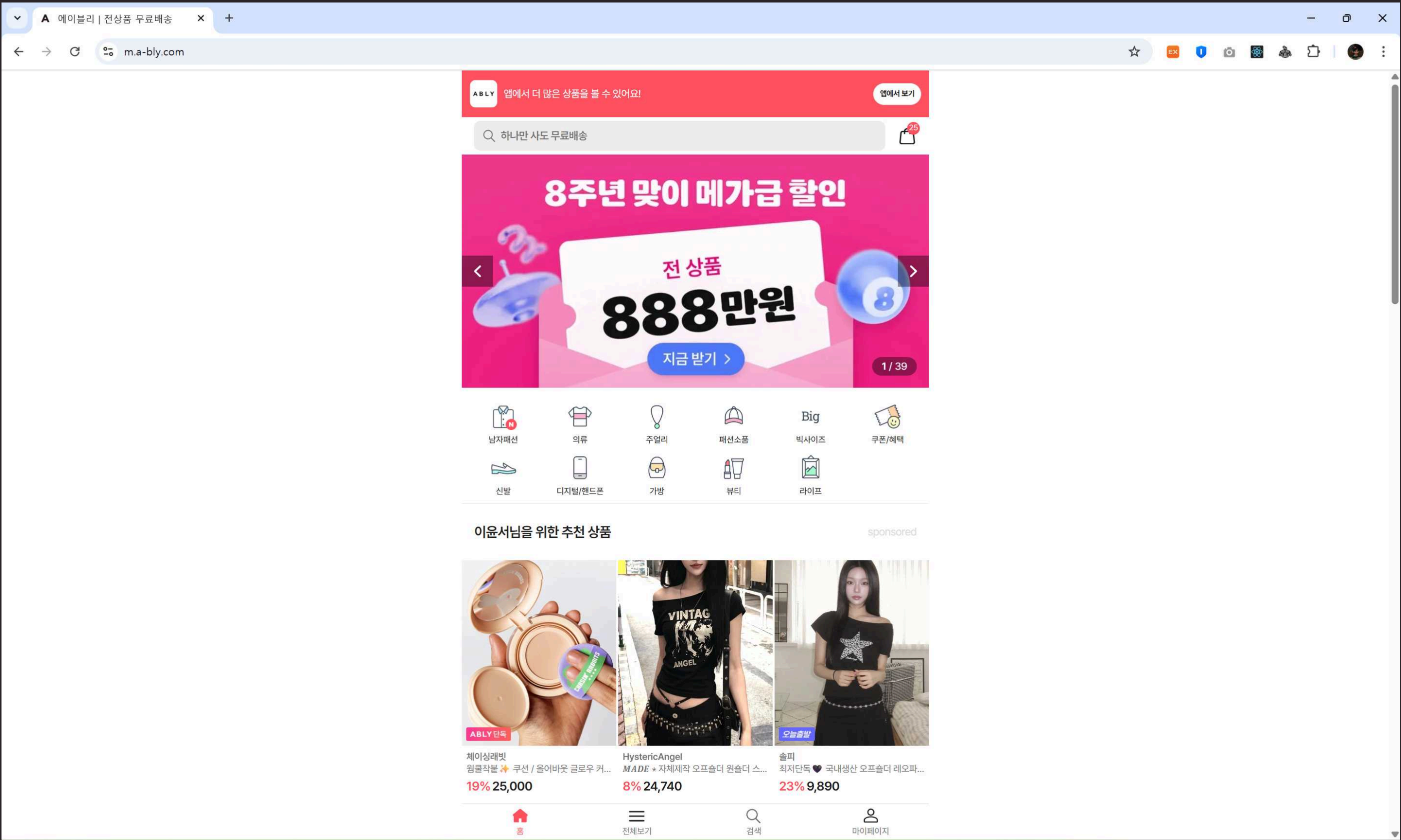
반응형 웹사이트는
하나의 URL로
모든 기기에서 동일한 콘텐츠를
제공하므로, 크롤링 효율성과
검색순위 개선에 기여

유지보수 용이성

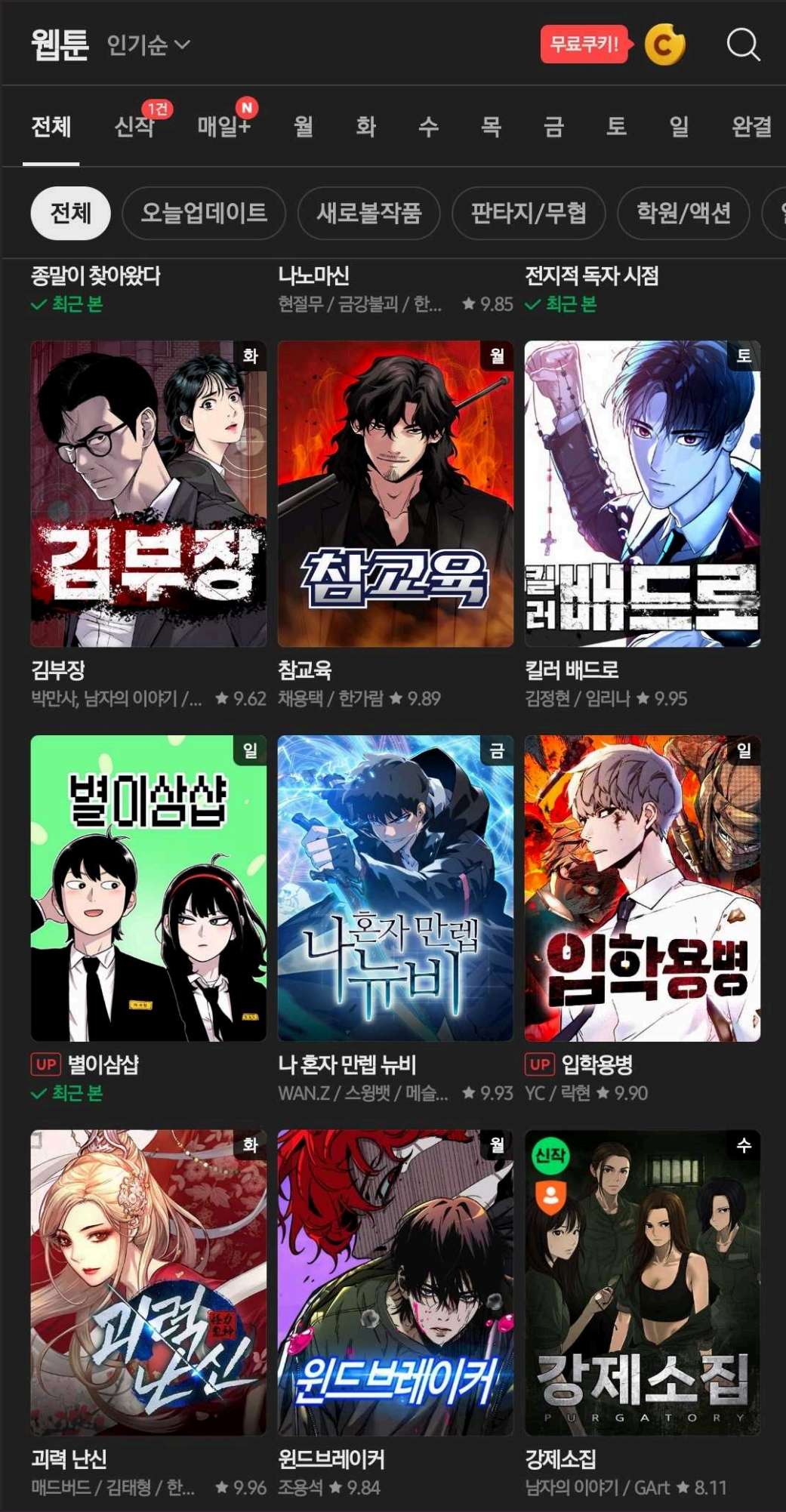
하나의 코드를 기반으로
모든 기기에서 작동하기 때문에
유지 보수가 훨씬 간편

추가비용 절감, 업데이트 속도 향상

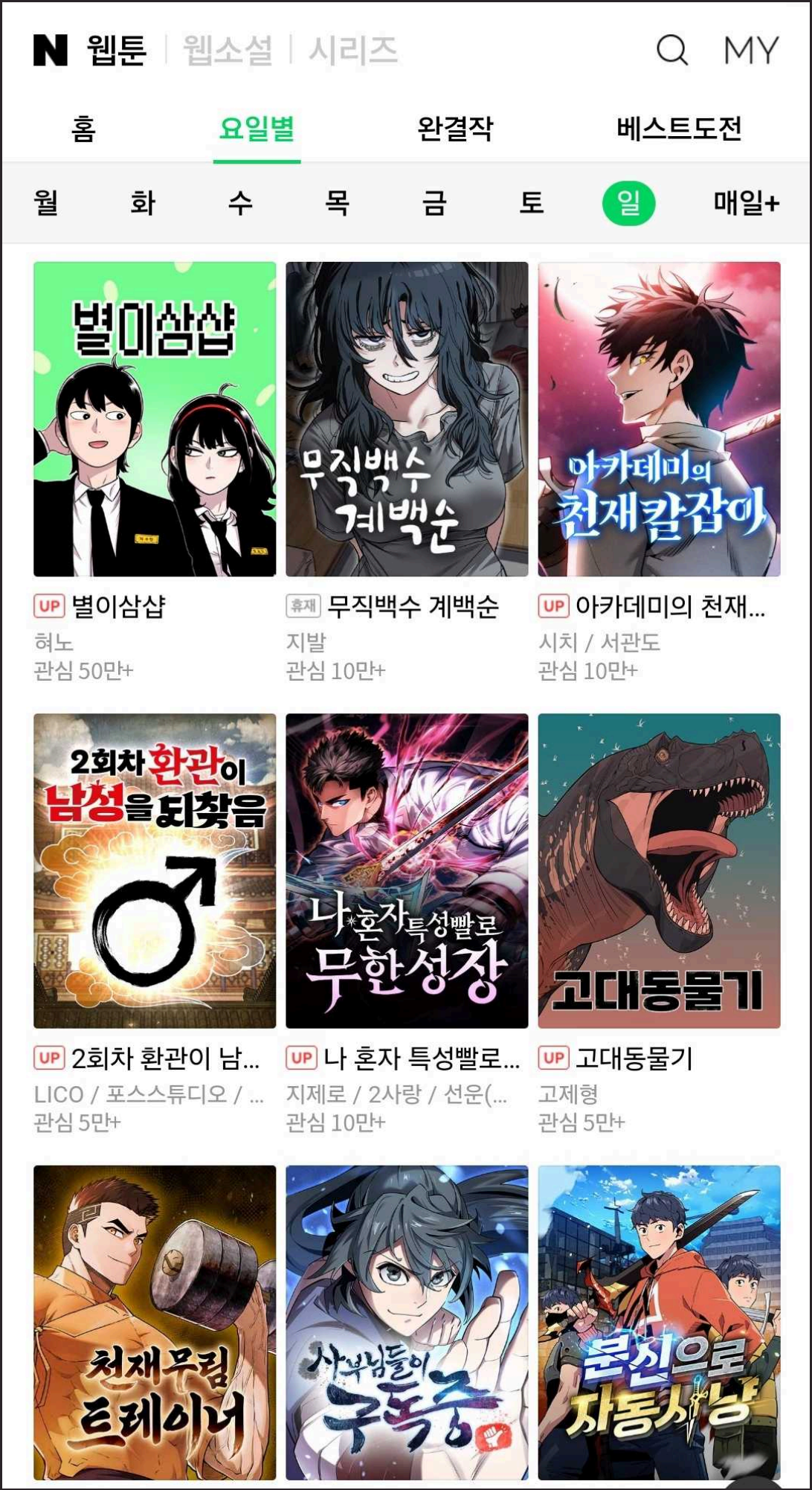
웹앱?



Native 앱



웹 앱



우리는 모바일 환경에서 사용하기에 적합한
앱 서비스를 만들고 싶은데

그럼 Native 개발 (Kotlin, Swift 등)을 따로 배워야한다고?
그러기는 싫은데...

→ 웹 앱 형태로 제작!

반응형 웹 구현하기

반응형 웹 구현하기

MEDIA QUERY

- 미디어 타입 : SCREEN, PRINT, SPEECH 등의 미디어 타입을 지정. 생략 가능
- AND : AND 키워드를 사용하여 미디어 타입과 미디어 특성을 조합할 수 있음.
- 미디어 특성 : 뷰포트의 크기, 해상도 기기 방향 등의 미디어 특성을 지정함.

ex) WIDTH, HEIGHT, MIN-WIDTH, MAX-WIDTH, ORIENTATION 등

- ONLY, NOT은 특정 기기에만 혹은 특정 기기는 포함하지 않도록 하는 옵션. 생략 가능

Media Type

Expression To Test

```
@media screen and (max-width:525px) {  
  img[class="hide"] {  
    display: none !important;  
  }  
}
```

Conditional CSS To Display

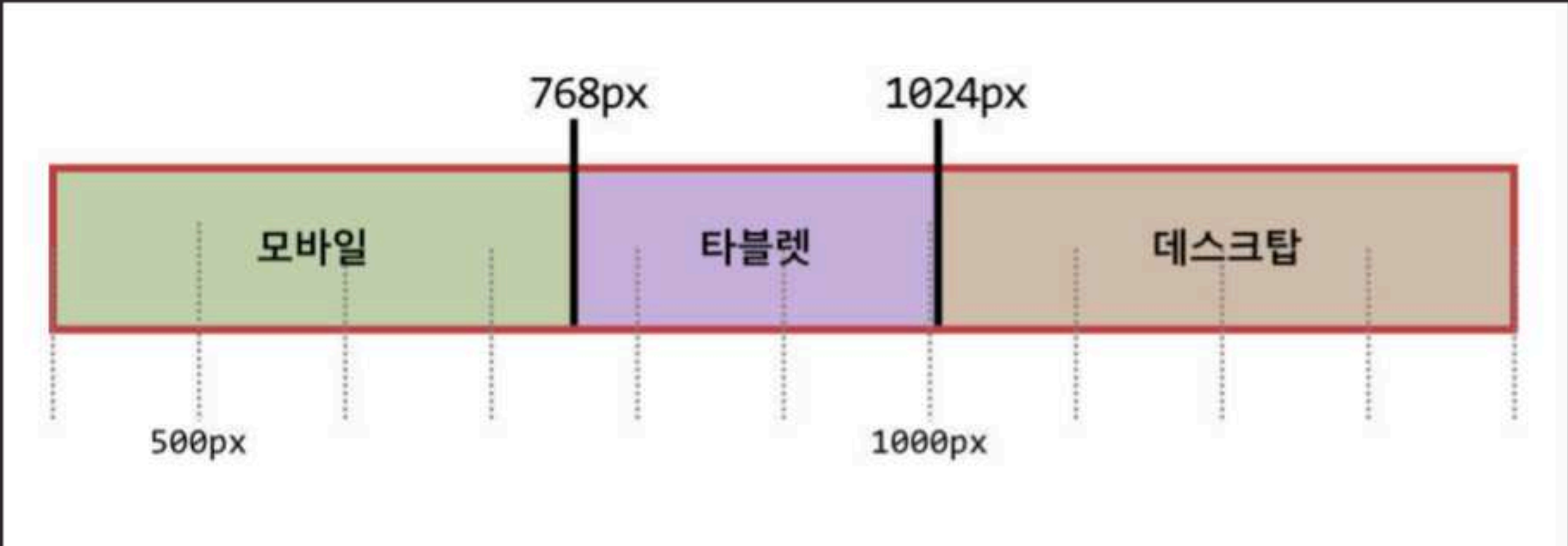
MEDIA QUERY

미디어쿼리 속성은 프로그래밍 언어처럼 연산자(<, >, =)를 사용하지 않으며, 속성 명 앞에 MIN-, MAX -를 붙여서 최소값과 최대값을 확인

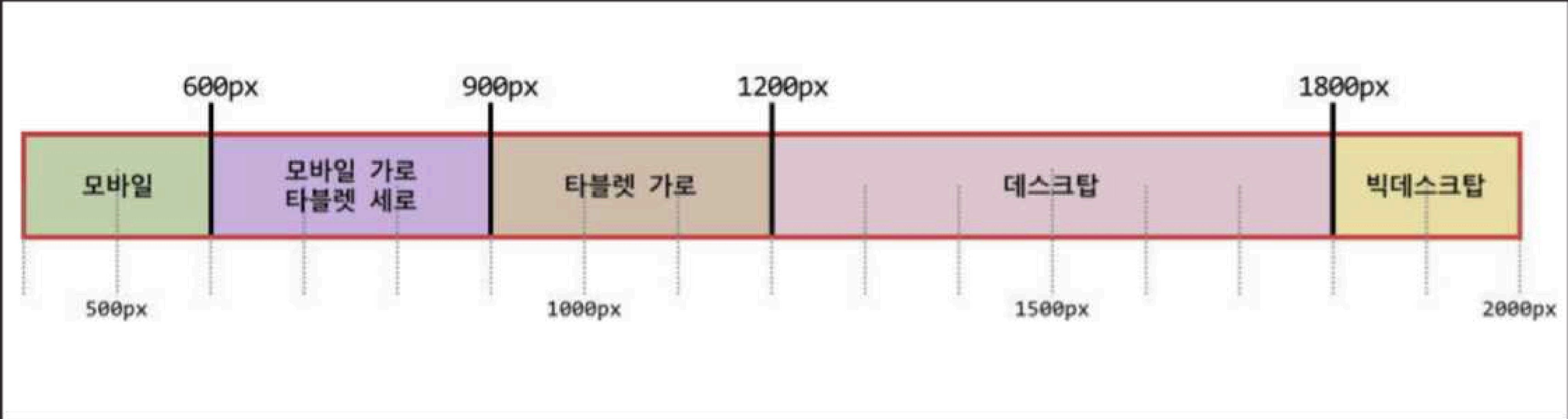
AND를 통해 나열할 수도 있고 MIN, MAX를 동시에 써서 범위를 나타내줄 수도 있음

- **width** : 웹페이지의 가로 길이를 판단합니다.
- **height** : 웹페이지의 세로 길이를 판단합니다.
- **device-width** : 기기의 실제 가로 길이를 판단합니다.
- **device-height** : 기기의 실제 세로 길이를 판단합니다.
- **orientation** : width의 height를 구하여 width의 값이 길면 landscape로, height 값이 길면 portrait로 판단합니다.

MEDIA QUERY 기준



모바일 ~ 599px
태블릿 600~1199px
데스크탑 1200px 이상



MEDIA QUERY 기준



```
/* 미디어 타입이 screen이며, 뷰포트의 최소 너비가 768px 이상일 때 */  
@media screen and (min-width : 768px) {  
    /*스타일*/  
}  
  
/* 미디어 타입이 print이며, 뷰포트의 해상도가 300dpi 이상일 때 */  
@media print and (min-resolution : 300dpi) {  
    /*스타일*/  
}  
  
/* 뷰포트의 최대 너비가 767px 이하이며, 기기 방향이 세로일 때*/  
@media (max-width : 767px) and (orientation : portrait) {  
    /*스타일*/  
}
```

MEDIA QUERY 기준

TailwindCSS v4 제공 반응형 스타일

Breakpoint prefix	Minimum width	CSS
`sm`	40rem (640px)	`@media (width >= 40rem) { ... }`
`md`	48rem (768px)	`@media (width >= 48rem) { ... }`
`lg`	64rem (1024px)	`@media (width >= 64rem) { ... }`
`xl`	80rem (1280px)	`@media (width >= 80rem) { ... }`
`2xl`	96rem (1536px)	`@media (width >= 96rem) { ... }`



```
<div className="bg-red-500 sm:bg-green-500 md:bg-blue-500 lg:bg-yellow-500">  
  뷰포트 크기에 따라 배경색이 바뀜  
</div>
```

MEDIA QUERY 기준

Tailwind 사용자 정의 중단점



```
/*index.css*/  
@import "tailwindcss" ;  
  
@theme {  
  --breakpoint-ph : 360px;  
  --breakpoint-dt : 1200px;  
}
```



```
<div className="dt:text-gray4 dt:top-[215px] ph:top-[67px] ph:text-gray6  
dt:body1 ph:body2">  
</div>
```


MEDIA QUERY 기준

react - responsive 라이브러리

각 Media Query 에 맞는
컴포넌트 렌더링 가능

```
npm install react-responsive
```

```
import {useMediaQuery} from 'react-responsive' ;
```

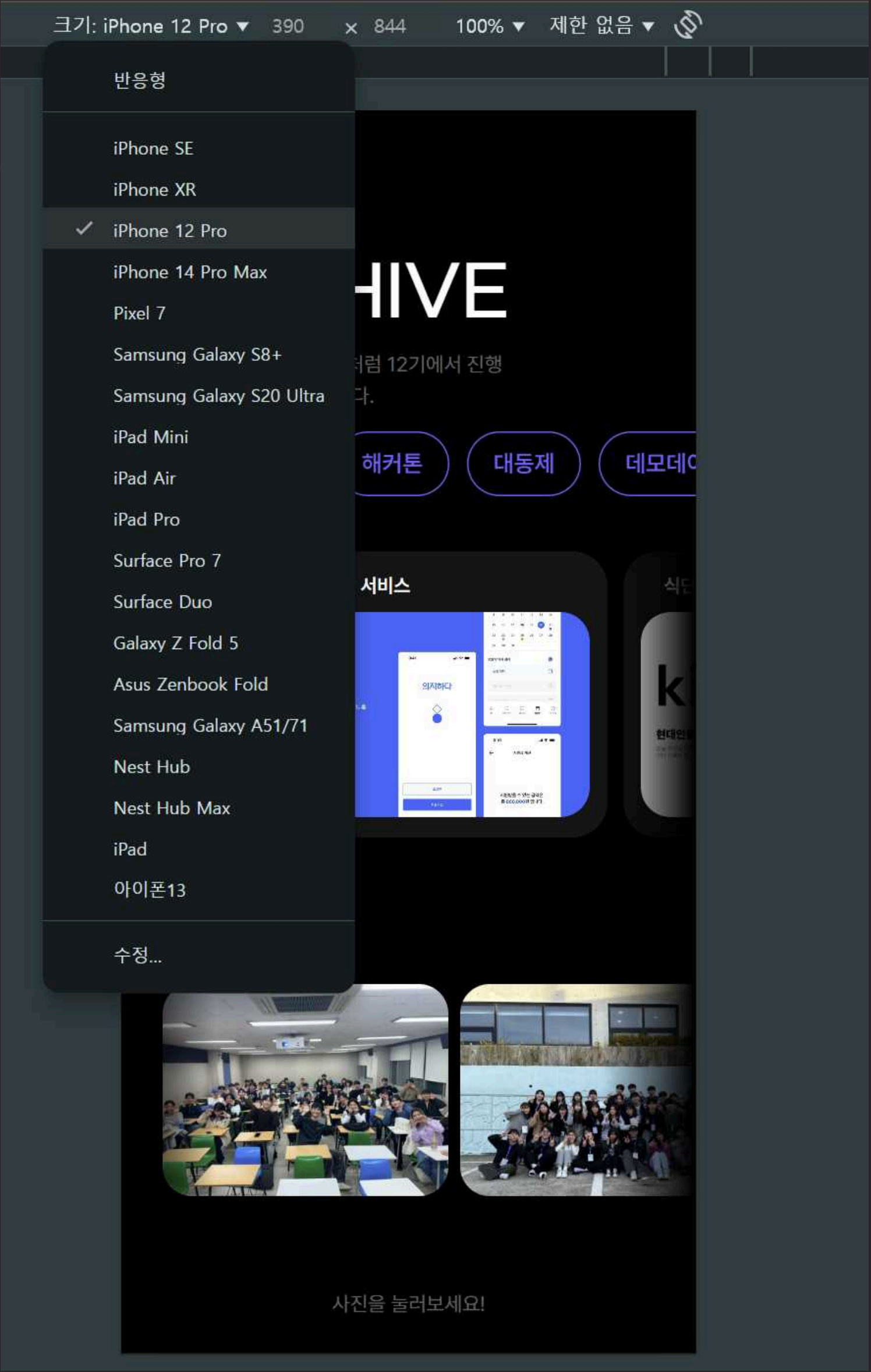
```
export const Mobile = ({children}) => {  
  const isMobile = useMediaQuery({  
    query : "(max-width:768px)"  
  });
```

```
  return <> {isMobile && children} </>  
}
```

```
export const PC = ({children}) => {  
  const isPc = useMediaQuery({  
    query : "(min-width:769px)"  
  });
```

```
  return <> {isPc && children} </>  
}
```

```
function App () {  
  return (  
    <>  
      <Mobile> 모바일 </Mobile>  
      <PC> PC </PC>  
    </>  
  )  
}
```

개발자 도구

직접 화면을 늘리거나 줄여서 반응형 디자인을 확인할 수도 있지만, 개발자 도구를 이용하면 좀 더 간편하게 개발이 가능함. 개발자 도구 상단에서 픽셀을 지정하여 화면을 보거나, 특정한 기기의 화면 너비 및 높이로 뷰를 확인할 수 있음.

reactCSS와 공통 스타일

resetCSS?

```
/* http://meyerweb.com/eric/tools/css/reset/
v2.0 | 20110126
License: none (public domain)
*/

html, body, div, span, applet, object, iframe,
h1, h2, h3, h4, h5, h6, p, blockquote, pre,
a, abbr, acronym, address, big, cite, code,
del, dfn, em, img, ins, kbd, q, s, samp,
small, strike, strong, sub, sup, tt, var,
b, u, i, center,
dl, dt, dd, ol, ul, li,
fieldset, form, label, legend,
table, caption, tbody, tfoot, thead, tr, th, td,
article, aside, canvas, details, embed,
figure, figcaption, footer, header, hgroup,
menu, nav, output, ruby, section, summary,
time, mark, audio, video {
  margin: 0;
  padding: 0;
  border: 0;
  font-size: 100%;
  font: inherit;
  vertical-align: baseline;
}
/* HTML5 display-role reset for older browsers */
article, aside, details, figcaption, figure,
footer, header, hgroup, menu, nav, section {
  display: block;
}
body {
  line-height: 1;
}
ol, ul {
  list-style: none;
}
blockquote, q {
  quotes: none;
}
blockquote:before, blockquote:after,
q:before, q:after {
  content: '';
  content: none;
}
table {
  border-collapse: collapse;
  border-spacing: 0;
}
```

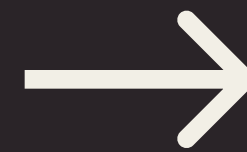
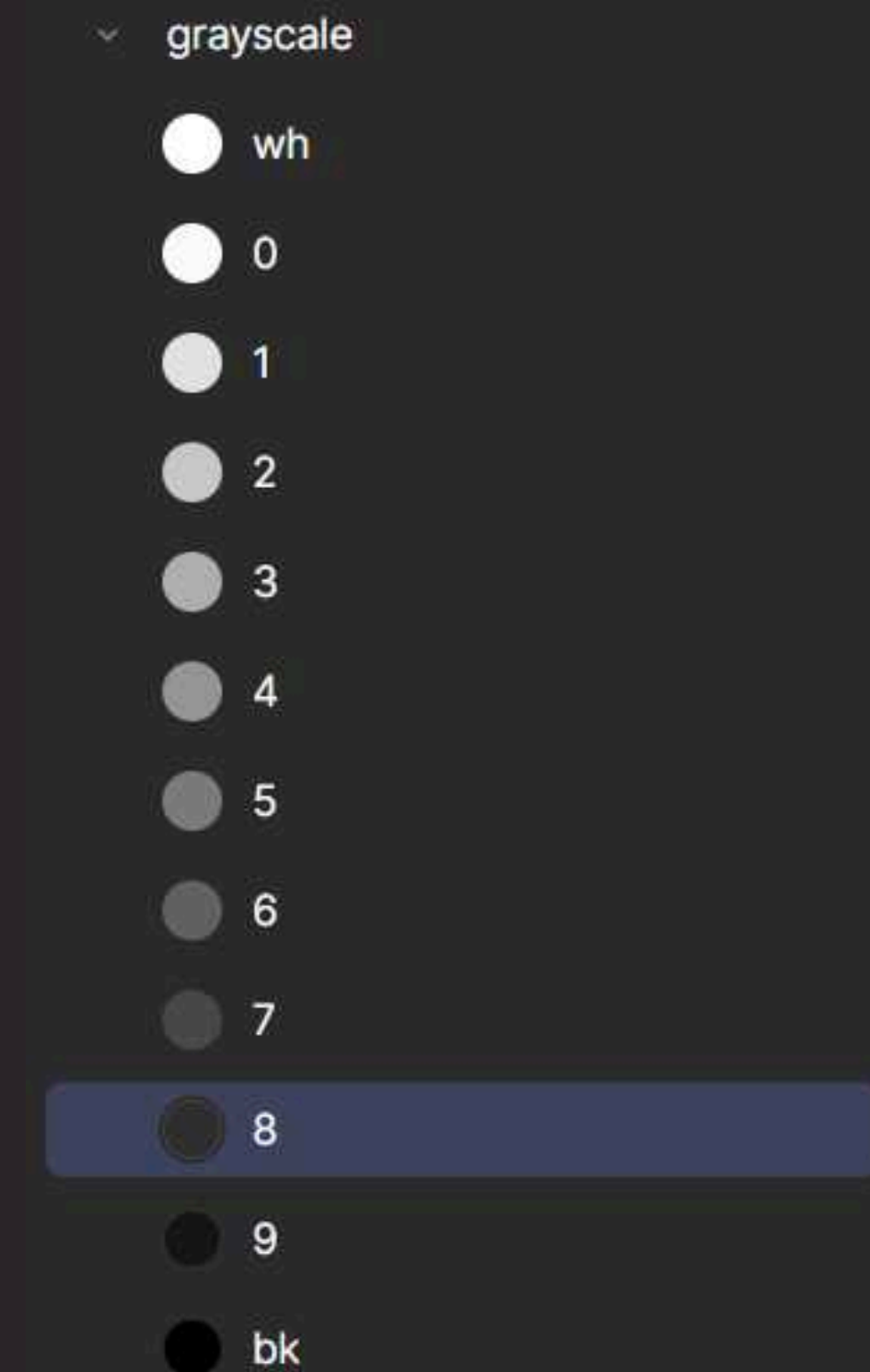


Preflight

<https://meyerweb.com/eric/tools/css/reset/>

공통 스타일 설정 - 색상

- Tailwind v4부터는 CSS에서 @theme 디렉티브를 사용해 색상을 직접 정의할 수 있음



```
@theme {  
  /* color */  
  --color-white: #ffffff;  
  --color-gray0: #fcfcfc;  
  --color-gray1: #e3e3e3;  
  --color-gray2: #c9c9c9;  
  --color-gray3: #b0b0b0;  
  --color-gray4: #969696;  
  --color-gray5: #7d7d7d;  
  --color-gray6: #636363;  
  --color-gray7: #4a4a4a;  
  --color-gray8: #303030;  
  --color-gray9: #171717;  
  --color-black: #000000;  
}
```

index.css

```
<p className="flex ph:gap-2 dt:gap-4">  
  <span className="text-gray0">모집 기간</span>  
  <span className="text-gray4">2025년 2월</span>  
</p>  
<p className="flex ph:gap-2 dt:gap-4">  
  <span className="text-gray0">모집 대상</span>  
  <span className="text-gray4">홍익대학교</span>  
</p>
```


공통 스타일 설정 - 폰트

Pretendard 폰트 사용 방법

<https://cactus.tistory.com/306>

1. src/index.html 에 CDN 추가

```
<!doctype html>
<html lang="en">
  <head>
    ...
    <link
      rel="stylesheet"
      as="style"
      crossorigin
      href="https://cdn.jsdelivr.net/gh/orioncactus/pretendard@v1.3.9/dist/web/static/pretendard.min.css"
    />
```

2. src/index.css @theme 적용

```
@import 'tailwindcss';

@theme {
  --font-pretendard: 'Pretendard', 'sans-serif';
  --font-gsans: 'GmarketSans', 'sans-serif';
}
```

3. font-pretendard로 사용!

```
export const HomePage = () => {
  return <div className="font-pretendard">HOMEPAGE</div>;
};
```

공통 스타일 설정 - 폰트

Pretendard 폰트 사용 방법

<https://cactus.tistory.com/306>

1. src/index.html 에 CDN 추가

```
<!doctype html>
<html lang="en">
  <head>
    ...
    <link
      rel="stylesheet"
      as="style"
      crossorigin
      href="https://cdn.jsdelivr.net/gh/orioncactus/pretendard@v1.3.9/dist/web/static/pretendard.min.css"
    />
```

2. src/index.css @theme 적용

```
@import 'tailwindcss';

@theme {
  --font-pretendard: 'Pretendard', 'sans-serif';
  --font-gsans: 'GmarketSans', 'sans-serif';
}
```

3. font-pretendard로 사용!

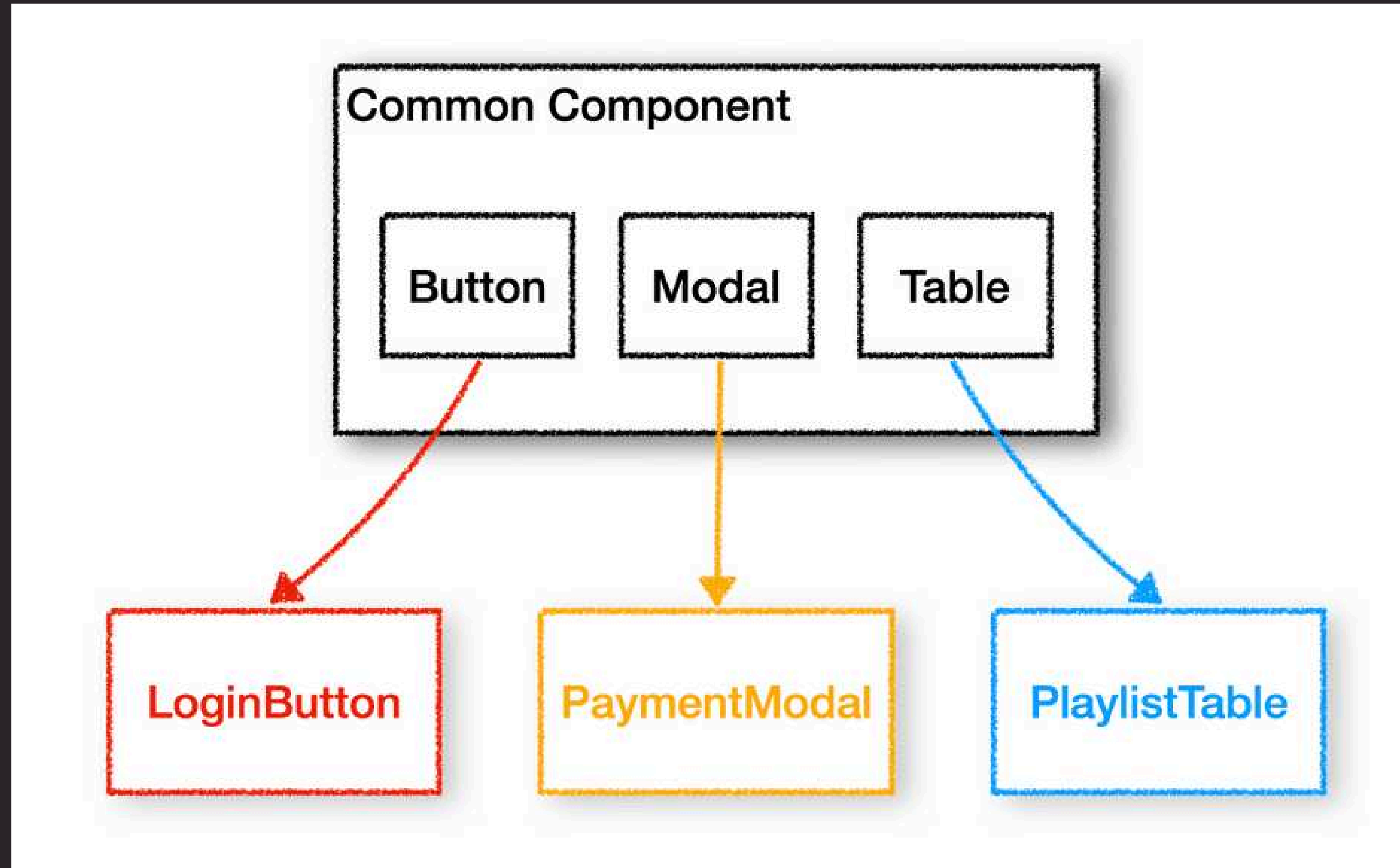
```
export const HomePage = () => {
  return <div className="font-pretendard">HOMEPAGE</div>;
};
```

공통 컴포넌트

공통 컴포넌트

- 프로젝트 내 여러 곳에서 반복적으로 사용하는 UI 요소를 하나의 컴포넌트로 추상화한 것
예: 버튼(Button), 카드(Card), 모달(Modal), 입력창(Input), 네비게이션 등

공통 컴포넌트 X	공통 컴포넌트 O
동일한 UI 요소를 매번 복붙	한 번만 정의하고 재사용 가능
디자인이 바뀌면 여러 파일을 고쳐야 함	한 곳만 수정하면 전체 반영됨
팀원이 새로운 컴포넌트를 만들 때 실수 가능성	일관된 디자인과 기능 유지 가능
생산성이 떨어짐	개발 속도 및 유지보수 효율 향상



규칙 ! - ex) 두 군데 이상에서 반복되면 공통 컴포넌트를 만든다.

공통 컴포넌트 - 버튼

```
import clsx from "clsx";

export default function Button({ children, variant = "primary", className,
...props }) {
  const baseStyle = "rounded-lg font-semibold transition-colors duration-
200";

  const variantStyle = {
    primary: "bg-blue-600 text-white hover:bg-blue-700",
    secondary: "bg-gray-100 text-gray-900 hover:bg-gray-200",
    outline: "border border-gray-400 text-gray-800 hover:bg-gray-50",
  };

  return (
    <button
      className={clsx(baseStyle, variantStyle[variant], className)}
      {...props}
    >
      {children}
    </button>
  );
}
```

반응형 추가

```
import clsx from "clsx";

export default function Button({ children, variant = "primary", className,
...props }) {
  const baseStyle = "rounded-lg font-semibold transition-colors duration-
200";

  const variantStyle = {
    primary: "bg-blue-600 text-white hover:bg-blue-700",
    secondary: "bg-gray-100 text-gray-900 hover:bg-gray-200",
    outline: "border border-gray-400 text-gray-800 hover:bg-gray-50",
  };

  return (
    <button
      className={clsx(baseStyle, variantStyle[variant], className)}
      {...props}
    >
      {children}
    </button>
  );
}
```


Netflix 사이트를 반응형 웹으로 구현하기

넷플릭스 사이트를 **반응형 웹**으로 디벨롭 해봅시다!

@theme를 사용하여 **breakpoint**를 설정하고
navbar 컴포넌트를 breakpoint에 따라 다르게 보이도록 해볼 거예요.
또한 **movieCard 컴포넌트**를 breakpoint에 따라 다르게 렌더링 해봐요!

제출물은 다음과 같습니다.

같이 만든 반응형 넷플릭스 사이트를 깃허브에 올리기